Smart Contract Audit Report

Contract: vulnerable_contract

Date: 2025-07-08 23:05:41

Risk Summary

Risk Score: 20/100

Risk Level: Low

Vulnerabilities Found: 2

Vulnerability Findings

1. Unchecked arithmetic operation

Severity: Unknown

Location: a + b

Description: Operation 'a + b' might cause overflow/underflow. Consider using safe math checks.

2. Missing validation in onTransfer

Severity: Unknown

Location: onTransfer

Description: onTransfer() should validate sender and amount to prevent exploit.

AI-Powered Analysis

Audit Summary for vulnerable_contract

Vulnerability 1: Unchecked Arithmetic Operation

Severity: Medium

Impact: Potential overflow or underflow of variables `a` and `b` can lead to unexpected behavior, potentially causing the contract to malfunction or allowing an attacker to manipulate the contract's state.

Potential Exploit Paths:

* An attacker can craft a scenario where the addition of `a` and `b` results in an overflow, causing the contract

to revert or behave unexpectedly.
* In a scenario where the contract uses the result of `a + b` to update a critical variable, an underflow could lead to an unintended state change.

Mitigation Strategies:

* Use safe math libraries, such as OpenZeppelin's SafeMath, to perform arithmetic operations. These libraries include built-in checks for overflow and underflow.
* Implement explicit checks for overflow and underflow before performing the addition operation.
* Consider using uint256 variables to minimize the risk of underflow.

Developer Advice: Always use safe math libraries or implement explicit checks for overflow and underflow when performing arithmetic operations in your smart contract. This will ensure that your contract behaves as expected and prevents potential exploits.

Vulnerability 2: Missing Validation in onTransfer

Severity: High
Impact: The lack of validation in the `onTransfer` function can allow an attacker to manipulate the contract's state by transferring arbitrary amounts or spoofing the sender's address.

Potential Exploit Paths:

* An attacker can call the `onTransfer` function with a malicious sender address or an excessive amount, potentially draining the contract's funds or causing unintended state changes.
* In a scenario where the contract relies on the `onTransfer` function to update critical variables, an attacker can exploit the lack of validation to manipulate the contract's state.

Mitigation Strategies:

* Implement explicit validation for the sender's address and the transferred amount in the `onTransfer` function.
* Use the `require` statement to ensure that the sender's address is not zero and the transferred amount is

within a reasonable range.

* Consider implementing a whitelist or blacklist for allowed sender addresses.

Developer Advice: Always validate user input and ensure that critical functions, such as `onTransfer`, have proper validation and access control mechanisms in place. This will prevent potential exploits and ensure the integrity of your contract's state.